Department of Electrical and Computer Engineering
University of Thessaly
ECE333 - Digital Systems Laboratory

Christos Karagiannis 3544

October 28, 2024

Laboratory Assignment $2^{\mathbf{nd}}$

Implementation of a Universal Asynchronous Receiver-Transmitter (UART) Module

Contents

1 Abstract **3**

2 Introduction **3**

3 Part A: Baud Rate Controller **4**
3.1 Implementation 4
3.2 Verification 5
3.3 Experiment/Final Implementation 5

4 Part B: Implementation of UART Transmitter **6**
4.1 Implementation 6
4.2 Verification 7
4.3 Experiment/Final Implementation 8

5 Part C: Implementation of UART Receiver **9**
5.1 Implementation 9
5.2 Verification 11
5.3 Experiment/Final Implementation 11

6 Part D: Implementation of UART Transmitter-Receiver for Serial Data Transfer **12**
6.1 Implementation 12
6.2 Verification 12
6.3 Experiment/Final Implementation 12
6.4 Optional Part: Advanced Use of the UART Module 13
6.4.1 Implementation 13
6.4.2 Verification 15
6.4.3 Experiment/Final Implementation 15

7 Conclusion **15**

1 Abstract

This document outlines the circuit description process of a serial communication system using the UART protocol. This includes the utilization of a Baud Controller, two Finite State Machines (FSMs)—one for the transmitter and one for the receiver—and ultimately the integration of these components into a universal top-level Module.

2 Introduction

The objectives of this assignment are the correct calculation of the waiting cycles required (based on the pre-agreed data transmission-reception speed) to achieve asynchronous communication, and the proper design and implementation of each Finite State Machine (FSM) on both the sender and receiver sides.

3 Part A: Baud Rate Controller

To achieve the asynchronous communication inherent to the protocol, an "agreement" on the rate at which one side sends information and the other receives it is necessary. Thus, we will implement a module where the input is the desired baud rate and the output is a pulse that will ultimately synchronize both sides.

3.1 Implementation

First, we will calculate the timing for each rate option using the formula $T_{SC} = \frac{1}{16 \cdot \text{Baud Rate}}$. The number 16 is due to the receiver operating 16 times faster than the transmitter. The required clock cycles are calculated using the relation $n = \frac{T_{SC}}{T_{clk}}$, where $T_{clk} = 10\text{ns}$.

Baud Select	Division result	Clock cycles
000	20833.333	20833
001	5208.333	5208
010	1302.083	1302
011	651.041	651
100	325.52	326
101	162.76	163
110	108.506	109
111	54.253	54

Table 1: Cycle calculations for the corresponding Baud Rate.

Having established the above, we can now create the Baud Controller, where an internal counter outputs a `sample_ENABLE` pulse and resets to zero as soon as it equals the selected cycle count.

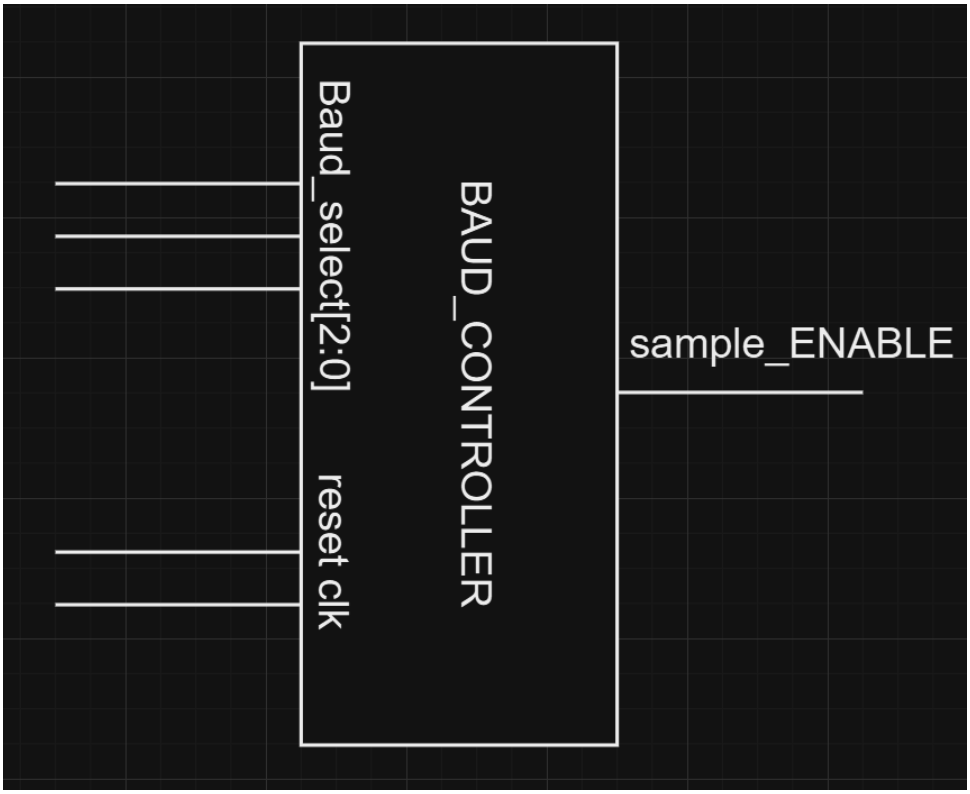

Figure 1: Dataflow of Part A.

However, since the counter takes discrete values and the clock period is constant, it is logical to have some error between our calculated values and the exact values our system can represent. The Relative Error is, therefore:

Baud Rate	Clock cycles	Calculated Baud Rate	Relative Error
300	20833	300.0048	$1.6 \cdot 10^{-5}$
1200	5208	1200.0768	$6.4 \cdot 10^{-5}$
4800	1302	4800.3072	$6.4 \cdot 10^{-5}$
9600	651	9600.6144	$6.4 \cdot 10^{-5}$
19200	326	19171.7791	0.00147
38400	163	38343.5582	0.00147
57600	109	57339.4495	0.00454
115200	54	115740.7407	0.00467

Table 2: Relative Error for the corresponding Baud Rate.

3.2 Verification

To verify the circuit's behavior, we wrote a testbench and selected every available baud select option after the `sample_ENABLE` output became 1.

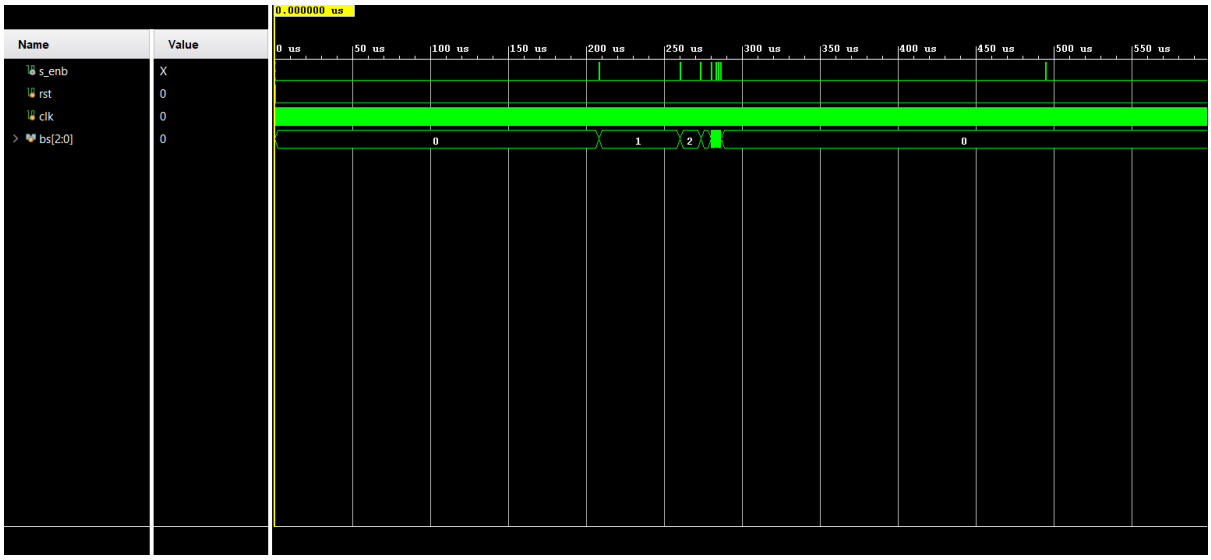

Figure 2: Waveforms of Part A.

3.3 Experiment/Final Implementation

The initial implementation worked; however, it utilized a memory block structure. Following a discussion with the teaching assistants, in order to avoid the use of unnecessary flip-flops, instead of using eight 15-bit registers, we now use a single register to store the appropriate number of cycles and compare it directly with the counter.

4 Part B: Implementation of UART Transmitter

Now that we have implemented the synchronization mechanism between the two ends, we can proceed to design the transmitter. The protocol followed by the transmitter FSM is as follows: it generally remains in an idle state, and if the upper layer wants to transmit data (the data length is 8 bits), it must send a total of 11 bits: 1 start bit, 8 `Tx_DATA` bits, 1 parity bit, and 1 stop bit.

4.1 Implementation

A 5-state Moore FSM was used to implement the transmitter. Following the above logic and delaying the transmitter 16 times (using a counter that increments with every pulse of the Baud Controller's output) relative to the receiver, this FSM has the following states:

- **TxIdle**: when the transmitter is in the idle state.
  - `TxD = 1`
  - `Tx_BUSY = 0`
- **TxStart**: when the transmitter sends the start bit.
  - `TxD = 0`
  - `Tx_BUSY = 1`
- **TxSend**: when the transmitter sends the byte selected by the upper layer.
  - `TxD = Tx_DATA[i]`
  - `Tx_BUSY = 1`
- **TxParity**: when the transmitter sends the parity bit.
  - `TxD = ^Tx_DATA`
  - `Tx_BUSY = 1`
- **TxStop**: when the transmitter sends the stop bit.
  - `TxD = 1`
  - `Tx_BUSY = 1`

  The transitions between states are handled as follows:

  - **TxIdle** → **TxStart**: `Tx_EN` must be 1 and `Tx_WR` must be 1 for one clock period.
  - **TxStart** → **TxSend**: `Tx_EN` must be 1 and `CHANGE_STATE_FLAG` must be 1.
  - **TxSend** → **TxParity**: `Tx_EN` must be 1, `CHANGE_STATE_FLAG` must be 1, and `MSG_END` must be 1.
  - **TxParity** → **TxStop**: `Tx_EN` must be 1 and `CHANGE_STATE_FLAG` must be 1.
  - **TxStop** → **TxIdle**: The transition occurs independently of the inputs and the intermediate logic.

  Every time the counter (mentioned above) reaches the value of 16, the `CHANGE_STATE_FLAG` becomes 1 and the FSM prepares to change state. When in the `TxSend` state, the entire byte must be transmitted before transitioning to the `TxParity` state. For this reason, in addition to the `CHANGE_STATE_FLAG`, we also use `MSG_END`, which takes the value of 1 when all data from the upper layer has been sent.

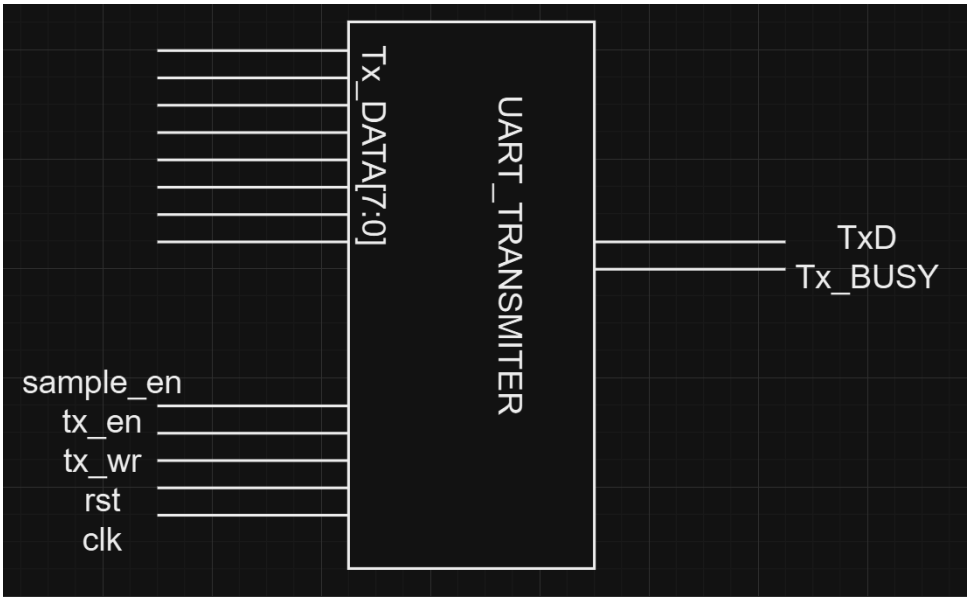

Figure 3: Dataflow of Part B.

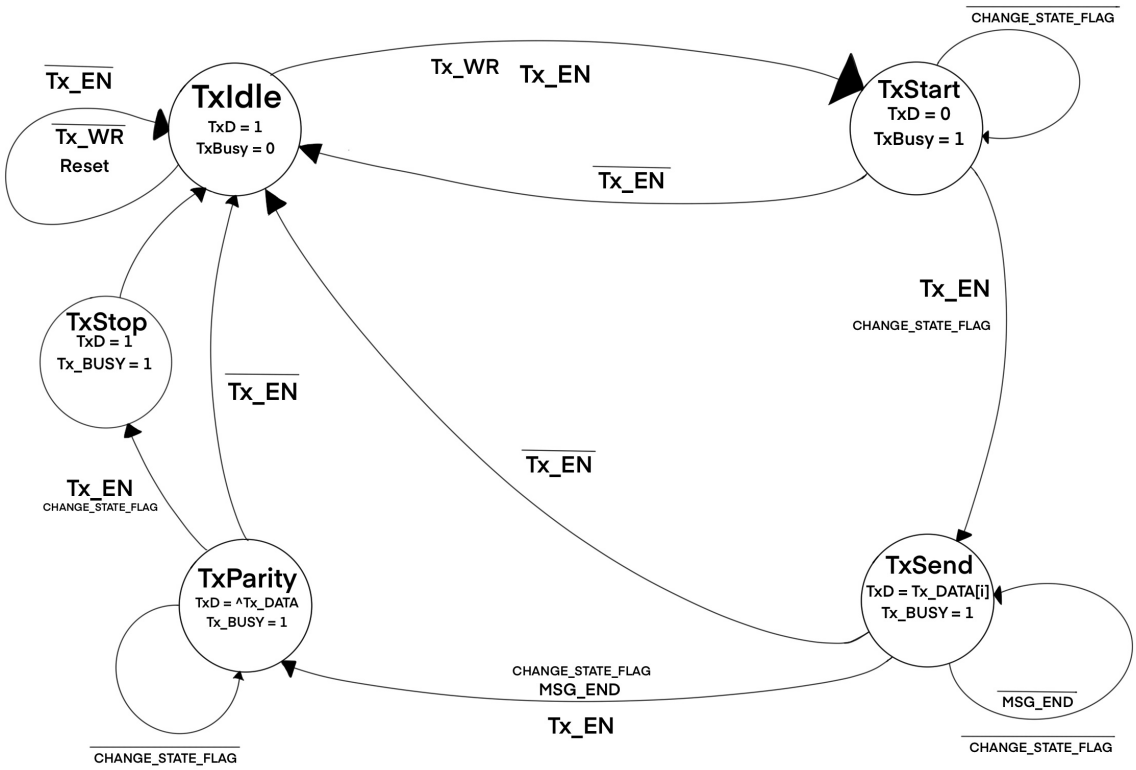

Figure 4: FSM of Part B.

4.2 Verification

To verify the behavior of the circuit, we wrote a testbench where we serially send the string "AA".

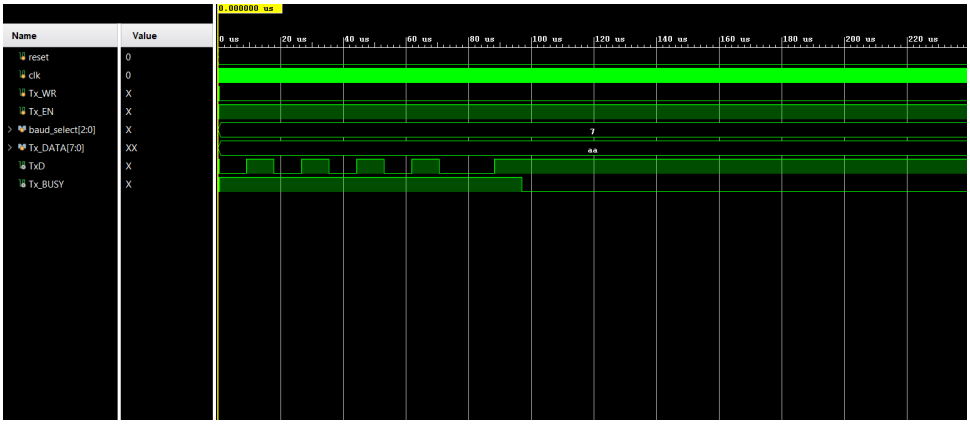

Figure 5: Waveforms of Part B.

4.3 Experiment/Final Implementation

The main issue I encountered was with the counter that counted 16 times to trigger the state change. This caused the transition to happen at the value of 17, which skewed the synchronization of the system. To fix this, I set the change to happen one `sample_ENABLE` pulse earlier, so now when the counter reaches 16, it resets to start over for the new state (or remains if we are in the middle of transmitting the data byte).

5 Part C: Implementation of UART Receiver

5.1 Implementation

For the implementation of Part C, we designed a 5-state Moore FSM where:

- **RxIdle**: when the receiver is in the idle state.
  - `Rx_DATA = FF`
  - `Rx_VALID = 0`
  - `Rx_PERROR = 0`
  - `Rx_FERROR = 0`
- **RxStart**: when the receiver receives the start bit.
  - `Rx_DATA = FF`
  - `Rx_VALID = 0`
  - `Rx_PERROR = 0`
  - `Rx_FERROR = 0/1` (1 is for the case of a frame error)
- **RxSample**: when the receiver receives the data string from the sender.
  - `Rx_DATA = FF`
  - `Rx_VALID = 0`
  - `Rx_PERROR = 0`
  - `Rx_FERROR = 0/1` (1 is for the case of a frame error)
- **RxParity**: when the receiver receives the parity bit.
  - `Rx_DATA = FF`
  - `Rx_VALID = 0`
  - `Rx_PERROR = 0/1` (1 is for the case of a parity error)
  - `Rx_FERROR = 0`
- **RxStop**: when the receiver receives the stop bit.
  - `Rx_DATA = DATA`
  - `Rx_VALID = 1`
  - `Rx_PERROR = 0`
  - `Rx_FERROR = 0`

The transitions between states are handled as follows:

- **RxIdle** → **RxStart**: `Rx_EN` must be 1 and `RxD` must be 0.
- **RxStart** → **RxSample**: `Rx_EN` must be 1 and `CHANGE_STATE_FLAG` must be 1.
- **RxSample** → **RxParity**: `Rx_EN` must be 1, `CHANGE_STATE_FLAG` must be 1, and `MSG_END` must be 1.
- **RxParity** → **RxStop**: `Rx_EN` must be 1 and `CHANGE_STATE_FLAG` must be 1.
- **RxStop** → **RxIdle**: The transition occurs independently of the inputs and the intermediate logic.

Similar to Part B, we use a counter that serves to synchronize us with the transmitter and dictates the sampling interval. We opted to sample each received bit 8 times, thereby increasing the likelihood of detecting a Framing Error, if one occurs.

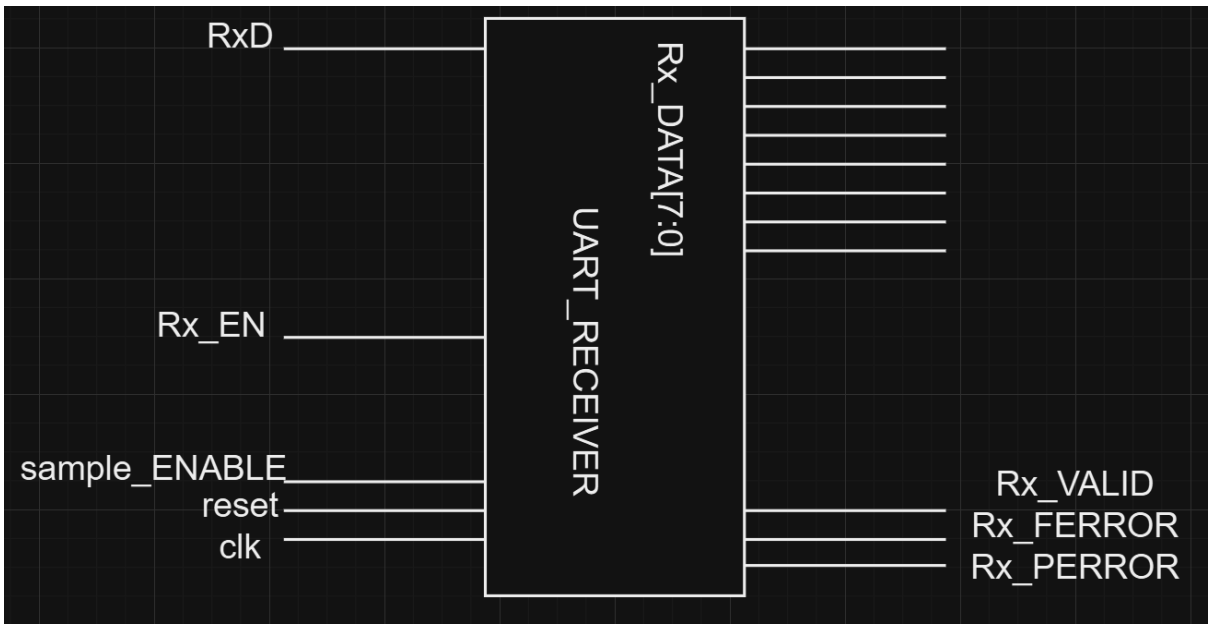

Figure 6: Dataflow of Part C.

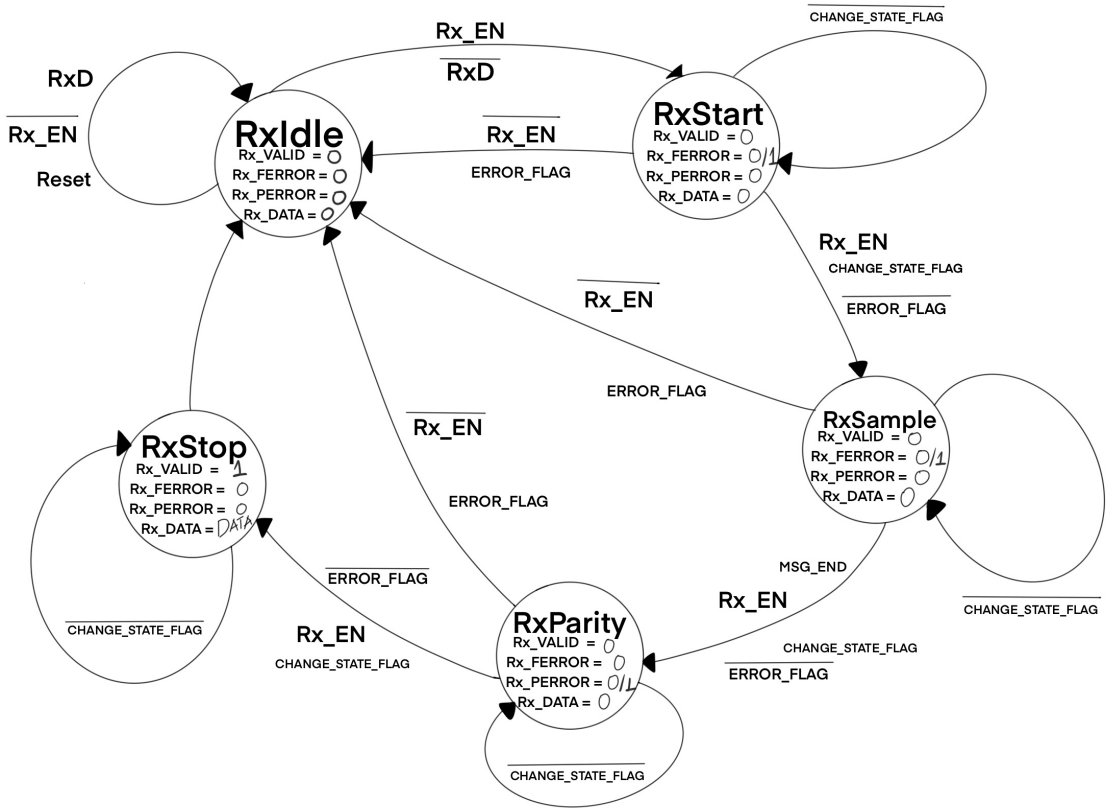

Figure 7: FSM of Part C.

5.2 Verification

To verify the circuit's behavior, we wrote a testbench to examine every possible scenario: the data being valid, the occurrence of a Parity Error (`Perror`), and the occurrence of a Framing Error (`Ferror`).

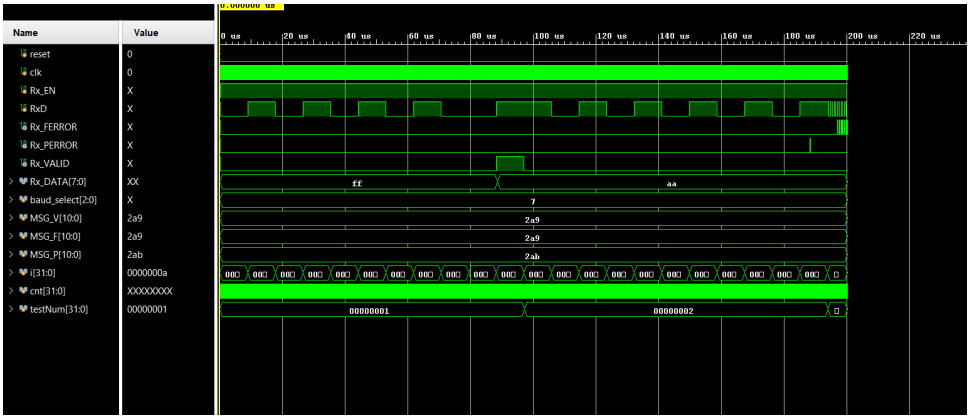

Figure 8: Waveforms of Part C.

5.3 Experiment/Final Implementation

Just like in Part B, there was an issue with the counter and the exact moment the `CHANGE_STATE_FLAG` should be asserted. There was also an issue with latches being inferred by the synthesizer because there were states where outputs of the module were left unassigned. Nevertheless, by adjusting the counter to set `CHANGE_STATE_FLAG` to 1 at the value of 15, and thoroughly covering all assignment cases in the combinational circuit, everything worked smoothly.

6 Part D: Implementation of UART Transmitter-Receiver for Serial Data Transfer

6.1 Implementation

For Part D, all that is left to do is properly connect the above modules and test the communication between them.

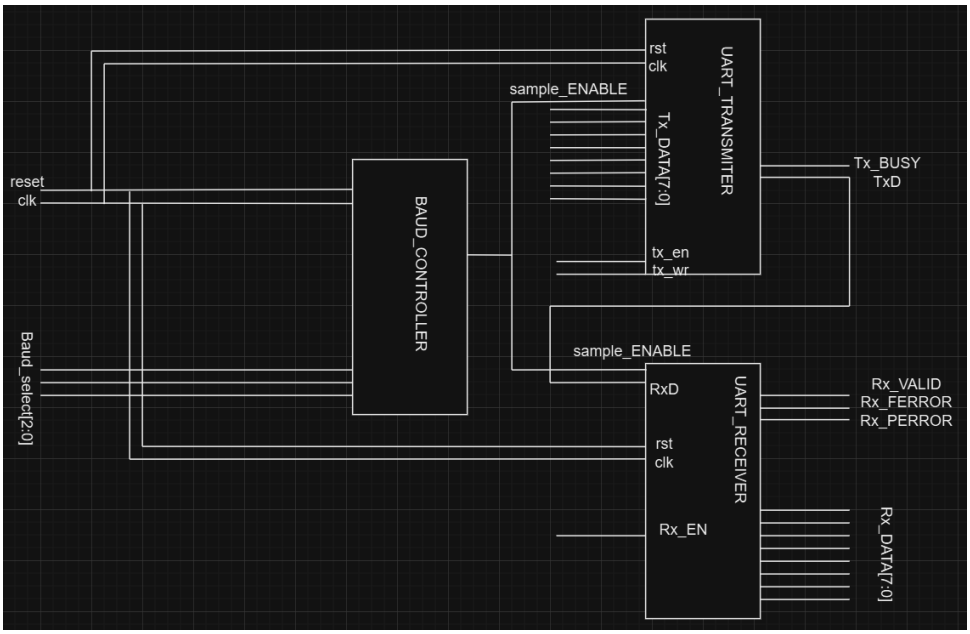

Figure 9: Dataflow of Part D.

6.2 Verification

To verify the behavior of the circuit, we wrote a testbench where we feed the transmitter the data we want to send every time it is not busy, and the receiver increments the read address for the transmitter every time the received data is valid.

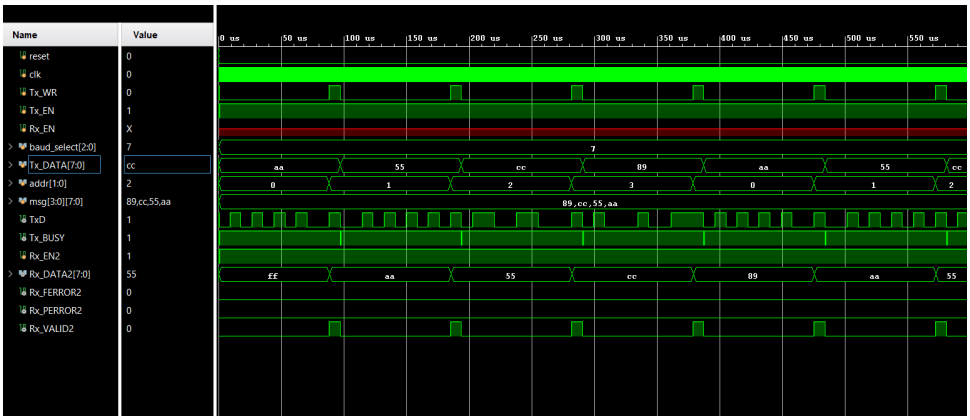

Figure 10: Waveforms of Part D.

6.3 Experiment/Final Implementation

All issues encountered were already mentioned in the previous sections. Addressing and debugging those issues in their respective parts led to immediate success in the experimental phase of Part D.

6.4 Optional Part: Advanced Use of the UART Module

Since the communication circuit has been designed and proven by Vivado to yield a synthesizable circuit, for the optional part, the only required step is designing one more FSM for improved user operation. We also slightly modified the module from the first assignment so it now accepts the characters we want to display as inputs, determines which anodes are activated, and utilizes all 8 light displays, since the largest Baud Rate requires 6 of them.

6.4.1 Implementation

To implement the Optional Part, we created a 4-state Moore FSM, where:

- **s_IDLE**: upon circuit initialization or returning from the `s_RUN` state.
  - `an0-7 = 8'h00`
  - {`abcdefg`} `= IDLE_MSG`
  - `Tx_BUSY = 0`
  - `UART_PERROR = 0`
  - `UART_FERROR = 0`
  - `UART_VALID = 0`
- **s_BS**: when selecting the baud rate.
  - `an0-1 = 2'b00`
  - `an2-7 = 6'b111111`
  - {`abcdefg`} `= baud_select`
  - `Tx_BUSY = 0`
  - `UART_PERROR = 0`
  - `UART_FERROR = 0`
  - `UART_VALID = 0`
- **s_MSG**: when sending the data string.
  - `an0-1 = 2'b00`
  - `an2-7 = 6'b111111`
  - {`abcdefg`} `= Tx_DATA`
  - `Tx_BUSY = 0`
  - `UART_PERROR = 0`
  - `UART_FERROR = 0`
  - `UART_VALID = 0`

- **s_RUN**: when running the UART communication.
  - `an0-1 = 2'b00`
  - `an2-7 = 6'b111111`
  - {`abcdefg`} `= Rx_DATA`
  - `Tx_BUSY = 0/1` (the RGB LED will light up red while we are sending data).
  - `UART_PERROR = Rx_PERROR` (the RGB LED will light up blue).
  - `UART_FERROR = Rx_FERROR` (the RGB LED will light up red).
  - `UART_VALID = Rx_VALID` (the RGB LED will light up green).

The transitions between states are handled as follows:

- **s_IDLE** → **s_BS**: `db_btn_out` is 1.
- **s_BS** → **s_MSG**: `db_btn_out` is 1.
- **s_MSG** → **s_RUN**: `db_btn_out` is 1.
- **s_RUN** → **s_IDLE**: `db_btn_out` is 1.

The button is used to easily cycle through the states.

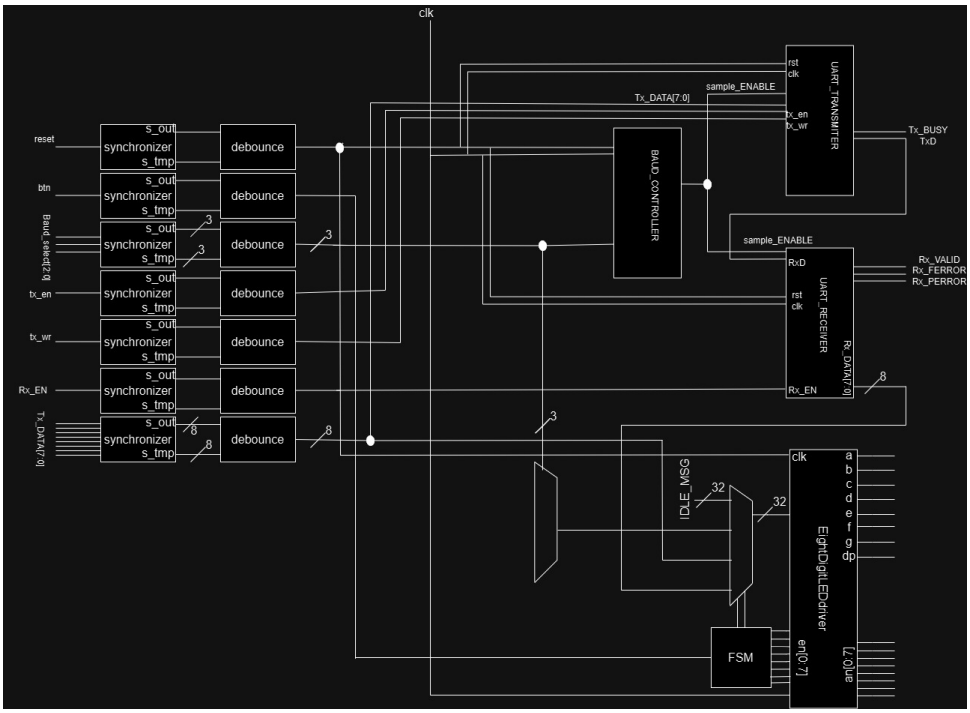

Figure 11: Dataflow of the Optional Part.

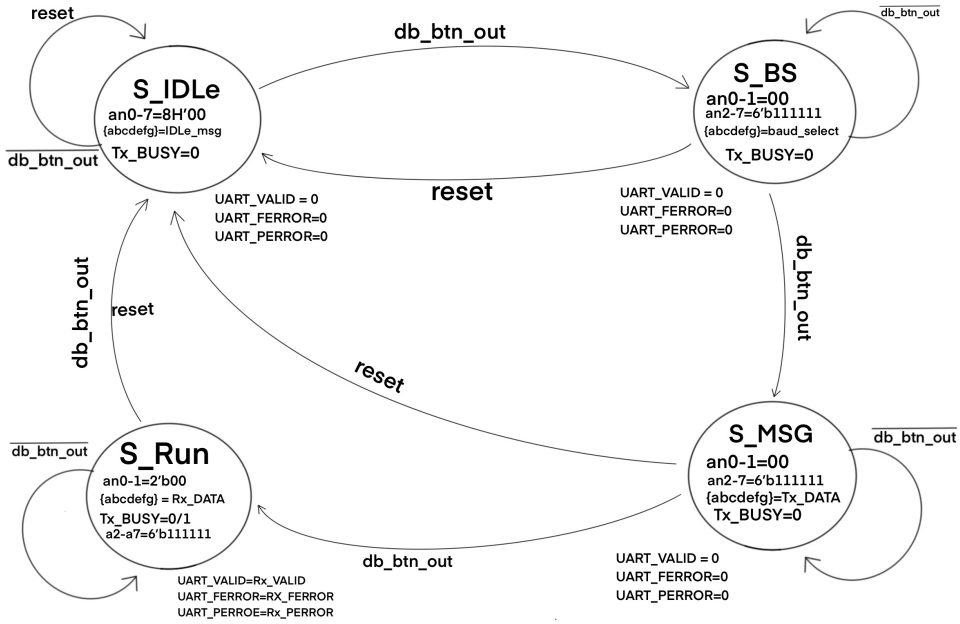

Figure 12: FSM of the Optional Part.

6.4.2 Verification

To verify the circuit's behavior, we extended the logic from Part D by simply introducing the button's logic.

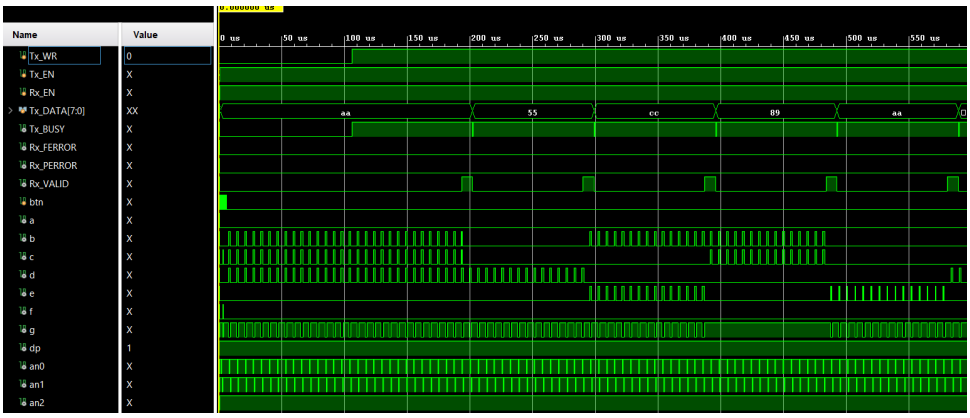

Figure 13: Waveforms of the Optional Part.

6.4.3 Experiment/Final Implementation

When utilizing the button, it was necessary to include an `always` block that stores the previous state of the button. This ensures a single pulse is generated, eliminating the possibility of multiple unintended state transitions when pressing or releasing the button.

7 Conclusion

Through this assignment, the design and utilization of FSMs was heavily comprehended, alongside the crucial separation of sequential and combinational logic for proper implementation. The optional part of the assignment was also significant, verifying the fact that careful design and modular independence of each circuit block are essential, so that the code can be minimally modified and still function seamlessly.